

CARVE-VLA: A Compute-Adaptive Agentic Runtime for Reliable Long-Horizon VLA Execution

Anonymous Authors

Anonymous Institution

Abstract. Vision-language-action (VLA) policies provide strong local manipulation skills, but long-horizon deployment requires both execution supervision and a feasible control-loop latency. Agentic wrappers can monitor progress and recover from failure, yet their additional inference and verification can make this timing problem worse. We present CARVE-VLA, a model-agnostic runtime that co-designs an Agentic Harness with an Optimize Runtime around a frozen VLA. The Harness provides typed action and capability contracts. It monitors execution risk and manages bounded recovery, memory, and auditable escalation. The optimizer calibrates flow steps and action commitment, fidelity-gates profiles, and contract-checks padding-view removal.

On LIBERO-10, the Agentic system improves a strong `pi05_libero` baseline from 90.0% (180/200) to 92.5% (185/200), with its dominant weak task improving from 55.0% to 75.0%. A paired 15-episode calibration preserves 14/15 successes while reducing mean VLA-call latency by $2.46\times$ and episode wall time by 32.3%. On an RTX 4090, compiled BF16 reduces deployment P95 from 159.59 to 67.40 ms with zero 80-ms policy-call misses. Static masked-view elision lowers P95 further to 56.19 ms, passes 45 replay states, and is non-inferior in a ten-episode gate (8/10 vs. 7/10). A W8A16 profile passes replay fidelity but loses a closed-loop recovery outcome and is therefore rejected. Finally, a ten-episode asynchronous gate reduces deadline misses from 417/3931 to 48/4287, but success falls from 7/10 to 5/10 under both tested schedules, so neither is promoted. Reliable Agentic control and efficient VLA deployment must therefore be validated jointly. On a second autoregressive OpenVLA-7B backend, prewarmed LLM compilation reduces P95 from 310.92 to 231.84 ms with exact paired actions, while generic INT8/NF4 profiles save memory but are slower and fail action fidelity.

Keywords: Vision-language-action models · Agentic policy · Robot manipulation · Long-horizon control · Efficient inference

1 Introduction

Vision-language-action (VLA) policies couple perception, language grounding, and low-level control within a single action model. RT-1 and RT-2 demonstrated large-scale visuomotor policies and web-knowledge transfer to robot control [2, 3].

Open X-Embodiment, Octo, OpenVLA, $\pi_{0.5}$, and $\pi_{0.5}$ -style policies have further made generalist robot policies increasingly accessible [18, 17, 11, 1, 20]. Once such policies become strong on short-horizon skills, the next question is no longer whether a robot can execute one local behavior, but whether it can organize many local behaviors into a reliable long-horizon process.

Long-horizon manipulation exposes failures that are easy to miss in atomic tasks. Benchmarks such as CALVIN and LIBERO require skill chaining, temporal coherence, and recovery from off-nominal states [15, 14]. Recent long-horizon VLA analyses make the same point: stronger action models improve local competence, yet subtask handoff, accumulated drift, and recovery remain persistent bottlenecks [6, 9]. For a strong fine-tuned VLA, many residual failures are therefore better described as missing *process control* rather than missing low-level control. The policy may know how to reach, grasp, or place, but the execution loop may not know when progress has stalled, when a transition motion is needed, when a task-specific prior should be injected, or when the current attempt should be retried.

This paper frames the issue as an *execution-supervision problem* for frozen VLAs:

How should a strong frozen VLA be embedded in a monitored, recoverable, and realtime-aware long-horizon execution loop?

The framing matters because it separates two roles that are often conflated. The VLA is the local action generator. The runtime is responsible for progress monitoring, context injection, recovery, call scheduling, and evidence logging. This separation is especially relevant when the available compute budget cannot support retraining a foundation-scale robot policy, but can support a careful control layer around it.

We instantiate this idea as CARVE-VLA, a Compute-Adaptive Agentic Runtime for VLA Execution. Its Agentic Harness contains transition, memory/prior, critic/retry, deployable risk monitoring, and stateful physical-recovery components. Its Optimize Runtime prepares a hardware-bound inference profile and schedules the resulting policy online. The two layers meet only through typed observations, actions, capabilities, deadlines, and Agentic-event traces. This boundary lets the same Harness wrap another chunked VLA while preventing an unsupported optimization from being silently reported as active. All modules operate around the original policy server; the VLA backbone remains frozen.

Our contributions are:

- We formulate frozen-VLA deployment as a joint execution-supervision and control-latency problem, separating the Agentic Harness from a reusable Optimize Runtime through explicit action, capability, and trace contracts.
- We introduce deployable risk monitoring and staged escalation from replanning to bounded physical recovery, verification, semantic planning, or safe stop, without simulator privileged state.
- We develop a hierarchical deployment gate. It calibrates flow steps and action commitment independently, then uses replay and closed-loop tests to reject distorted quantized policies and unsafe asynchronous schedules.

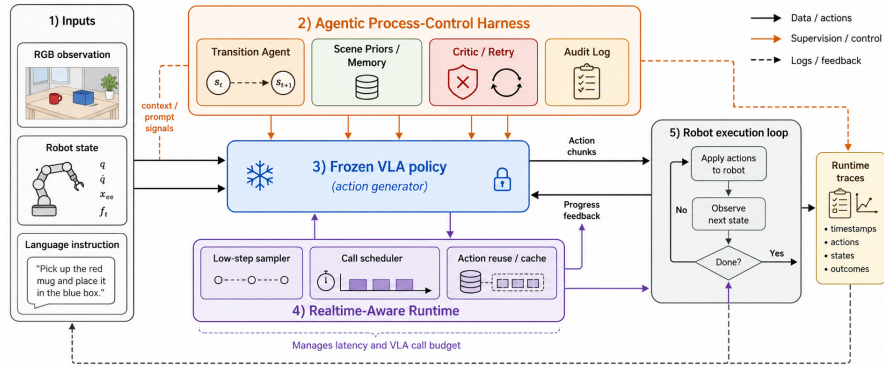


Fig. 1. Overview of CARVE-VLA. A frozen VLA remains the action generator. The Agentic Harness supervises execution and recovery, while the Optimize Runtime calibrates and schedules policy inference under an explicit control deadline.

- We provide a completed LIBERO-10 Agentic evaluation and paired runtime evidence on $\pi_{0.5}$, plus a cross-family OpenVLA-7B gate that separates faithful decode acceleration from rejected generic low-bit deployment.

2 Related Work

Foundation VLA policies. Modern VLA research has progressed from RT-1 and RT-2 to open and generalist policy stacks such as Octo, OpenVLA, $\pi_{0.5}$, and $\pi_{0.5}$ [2, 3, 17, 11, 1, 20]. Open X-Embodiment and related efforts emphasize shared datasets and cross-robot policy learning [18]. These models make frozen or lightly adapted VLA policies strong enough that remaining failures become more diagnostically useful: when local skills are present, long-horizon performance depends increasingly on execution organization.

Long-horizon benchmarks. CALVIN and LIBERO evaluate language-conditioned manipulation across temporally extended tasks [15, 14]. Long-VLA and Robo-Cerebra further emphasize that temporal consistency, state transitions, and off-nominal recovery remain difficult even for strong policies [6, 9]. Our main evaluation uses LIBERO because it provides official interactive rollouts, task-level success signals, and a challenging but reproducible long-horizon suite.

Agentic augmentation and memory. Agentic robot systems augment policy execution with planners, verifiers, memory, tools, or online adaptation [25, 12, 19, 10]. Embodied memory work studies how short-term and long-term memory can support manipulation [22, 21]. CARVE-VLA is aligned with this direction but takes a conservative stance: modules remain lightweight, execution-time, and attributable to rollout events rather than absorbed into a new monolithic planner.

Failure detection and efficient inference. Failure-aware systems use VLMs or specialized monitors to detect execution errors, reason about failures, and trigger recovery [4, 7, 13]. ECoT and Fast ECoT show that explicit embodied reasoning can also improve action generation and efficiency [26, 5]. Realtime robot control imposes constraints that are not captured by success rate alone. Recent efficient VLA work studies caching, low-latency decoding, and realtime diffusion-policy execution [23, 8, 16, 24]. Our runtime study is complementary: instead of treating kernel speed, low-step sampling, or quantization as sufficient in isolation, we require action-level fidelity and paired closed-loop behavior before a profile can serve the Agentic execution loop.

3 Method

3.1 Frozen Policy and Chunked Execution

The baseline policy is a frozen `pi05_libero` policy derived from the $\pi_{0.5}$ family and served through the OpenPI stack. At control step t , the policy receives visual observations o_t , robot state s_t , and language instruction ℓ , and predicts an action chunk

$$A_t = \pi_\theta(o_t, s_t, \ell) = (a_t, a_{t+1}, \dots, a_{t+H-1}). \quad (1)$$

The executor applies part of the chunk before replanning. This scheme is effective for many short and medium-horizon tasks, but it can fail when locally plausible chunks do not smoothly connect adjacent subtask states. CARVE-VLA repairs this failure mode without changing π_θ : the backbone remains frozen, and external modules intervene only at inference time.

3.2 Execution-Supervision Loop

The key algorithmic object is the execution-supervision loop in Fig. 2. During normal progress, the system observes the state, queries the frozen VLA, executes a partial chunk, monitors progress, and continues. When the monitor detects stall, drift, or off-nominal progress, the runtime activates the agentic supervisor. The supervisor can insert a transition, retrieve scene priors or memory, request critic/retry, repair the prompt, and resume VLA execution. Every intervention is logged with state, action, context, outcome, and timing information.

3.3 Failure Taxonomy

CARVE-VLA is organized around a concrete failure taxonomy rather than a generic “reasoning helps” claim. We distinguish four recurring long-horizon failure modes: *transition failures*, where adjacent subtask phases do not connect smoothly; *contact or grasp failures*, where object interaction is locally unstable; *completion failures*, where the object is moved but the terminal condition is not

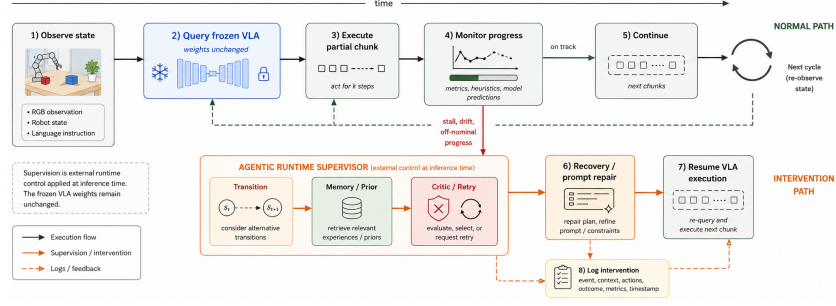


Fig. 2. Execution-supervision loop. Under normal progress, the frozen VLA continues chunked execution. Under stalled or off-nominal progress, an external runtime supervisor activates transition repair, memory/priors, or critic/retry, then resumes VLA execution and logs the intervention.

Table 1. Failure modes are tied to observable rollout evidence rather than post-hoc labels.

Failure mode	Observable signal	Helpful module	Expected ablation signal
Transition gap	low recent motion near a subtask boundary; repeated replans without phase change	Transition Agent	removing transition should hurt boundary-heavy tasks
Contact ambiguity	object is reached but grasp/place geometry remains unstable	Priors/memory	removing priors should hurt object/target-specific weak tasks
Incomplete rollout	object is manipulated but terminal condition is not achieved	Priors/memory or retry	task-specific hints or retry should improve weak tasks
Derailment	rollout state inconsistent with task progress after an intervention	Critic/Retry	retry count and recovery-to-success should rise when critic is active

reached; and *recovery failures*, where a failed segment is detected but the system cannot re-enter a useful state. Table 1 maps these failures to modules and expected evidence.

This taxonomy defines how the method should be falsified. If a module is claimed to repair a failure mode, the corresponding rollout signal should change in the logs or in a targeted ablation. For example, a useful Transition Agent should affect boundary-heavy weak tasks and appear in intervention counts, while a useful Critic/Retry module should produce nonzero retry events and recovery-to-success episodes. This is why we report the critic as under-activated rather than simply including it as a successful component of the full system.

3.4 Process-Control Modules

Fig. 3 summarizes the three process-control modules. The Transition Agent targets state-gap failures between adjacent subtask phases. It monitors recent motion and detects stalled progress when displacement over a sliding window falls below a threshold. Once triggered, it temporarily modifies the prompt with a transition instruction that requests a safe neutral motion and a reconnection maneuver. The VLA then generates a short intermediate chunk intended to move the robot to a more controllable state before normal task execution resumes.

The scene-prior and memory module injects lightweight structured hints based on task-relevant entities. Examples include suggested vertical offsets, approach direction, target-region bias, or grip-strength hints. The module is deliberately interpretable: instead of retrieving large document collections or training a memory policy, it maps object and scene keywords to compact priors and records episodic cues from successful executions. Its role is to reduce avoidable ambiguity at execution time and bias the frozen policy toward more stable interaction patterns.

The Critic/Retry module periodically evaluates whether execution appears stuck, incomplete, or inconsistent with the task objective. When a likely failure is detected, the module clears the current plan and issues a recovery or retry prompt tailored to the inferred error type. The intended behavior is not to replace the base policy, but to give it a better re-entry state after local derailment. In the current tuned configuration, critic-triggered retries are rare; we report this explicitly because it affects how the full system should be interpreted.

The online controller extends these semantic modules with a deployable physical risk monitor. It uses recent proprioceptive response, commanded motion, action age, uncertainty when available, and measured deadline slack; object poses and simulator state are excluded. A first confirmed stall requests a fresh VLA chunk. Repeated stall, slip, misgrasp, or contact can start a bounded retract, lift, and reobserve skill. Its normalized delta-Cartesian actions are checked against the policy action contract, executed in auditable phases, and followed by end-effector response verification. Only verified recovery permits a post-recovery VLA replan; exhausted budget escalates to a planner or safe stop. Terminal outcomes are stored in bounded episodic recovery memory.

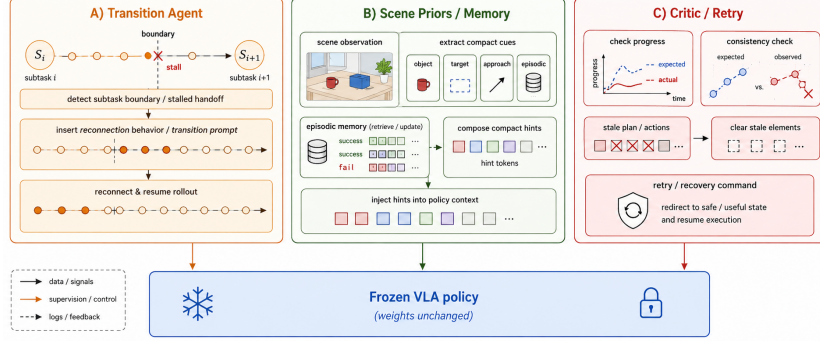


Fig. 3. Process-control modules in CARVE-VLA. Transition repair, scene priors/memory, and critic/retry target different observable rollout failures while leaving the VLA backbone unchanged.

3.5 Compute-Adaptive Realtime Runtime

Long-horizon recovery is useful only if the control loop remains feasible under robot timing constraints. CARVE-VLA separates offline deployment calibration from online scheduling. Offline, an *optimization profile* binds a model checkpoint and hardware identity to precision, backend, flow-step count, and committed action horizon. Candidate profiles first pass paired fixed-noise action checks for first-action error, chunk error, cosine similarity, gripper agreement, endpoint drift, and jerk. Surviving profiles then pass paired closed-loop states; replay fidelity alone cannot promote a quantized backend.

Online, the joint controller selects reuse, calibrated VLA inference, physical recovery, semantic escalation, or safe stop from risk and deadline evidence. Flow steps and commitment are not changed by an unvalidated stall heuristic. For low-risk execution, a single-flight prefetch can submit the next VLA call while the final two cached actions execute. The returned chunk is aligned by discarding its temporally covered prefix. Any physical-risk event, accurate replan, recovery, planner escalation, or safe stop invalidates the ticket; first inference and recovery-critical replans remain synchronous. This design optimizes end-to-end control latency rather than reporting model latency as a proxy for robot real time. A duty-cycle parameter can require fresh synchronous chunks between prefetches, allowing the closed-loop gate to evaluate both continuous and conservative schedules.

4 Experimental Design

4.1 Evidence Hierarchy

We separate four evidence tiers. The benchmark tier is the completed head-to-head `libero_10` comparison and targeted weak-task ablations. The calibration

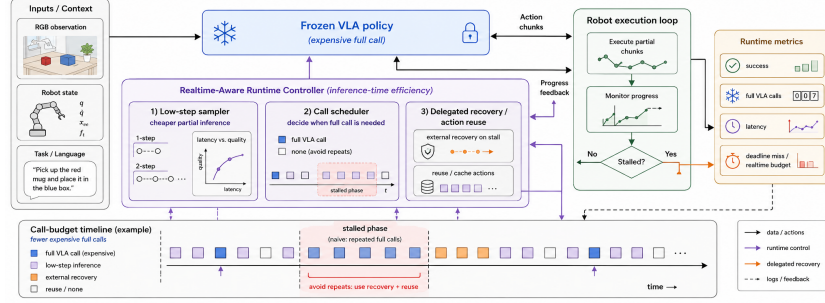


Fig. 4. Compute-adaptive realtime runtime. A calibrated backend bounds policy cost, while risk-aware scheduling, bounded recovery, action reuse, and asynchronous prefetch manage the full control critical path.

tier uses 15 paired T6/T8/T9 episodes to select flow steps and commitment without changing the frozen checkpoint. The deployment tier uses 45 fixed-noise replay observations and 50 timed calls per backend on an otherwise idle RTX 4090. Finally, the intervention tier restores identical failure states or initial states to test recovery and end-to-end scheduling. Simulator state is used only for evaluator-side restoration and labels, never by the online controller. This hierarchy prevents open-loop similarity, kernel timing, or a single successful recovery from being presented as benchmark success.

Table 2. Evaluation design and claim boundary.

Tier	Protocol	Role in paper
Benchmark	200 episodes/method libero_10	on long-horizon Agentic effectiveness
Calibration	15 paired T6/T8/T9 episodes	select flow steps and action commitment
Deployment	45 replay states; 50 timed calls/profile	fidelity, latency, VRAM, and deadline gate
Intervention	paired failures and T8/T9 states	held-out recovery and end-to-end real-time behavior

4.2 Metrics and Configurations

The main metric is task success from the simulator completion signal. Agentic runs additionally report interventions, recovery phase/outcome, action count, and episode length. Runtime metrics distinguish policy-call latency from the full control critical path, including observation preparation, simulator step, controller overhead, prefetch wait, deadline misses, wall time, and peak VRAM. Every

policy call records requested, applied, dropped, and fallback controls together with deployment-profile identity and the triggering Agentic event. Each reported runtime uses warmup and fixed noise; videos and JSONL traces are retained.

The baseline is the unmodified `pi05_libero` checkpoint. The original full Agentic system combines Transition, scene priors/memory, and Critic/Retry; follow-up diagnosis changes only control-side prompts and thresholds. Runtime experiments use the same PyTorch checkpoint. The fidelity-first profile uses two flow steps, ten committed actions, compiled BF16, and synchronous control. When an adapter guarantees a permanently padded camera slot, static masked-view elision (SMVE) asserts its all-false mask and removes it before visual encoding and prefix-KV construction. The real-time profile adds two-action, risk-gated prefetch; its causal control uses synchronous commit eight because both execute eight newly predicted actions per aligned chunk.

5 Results

5.1 Main LIBERO-10 Comparison

Table 3 shows the main completed comparison. The verified baseline reaches 90.0% (180/200), indicating that the frozen policy is already strong. Refined full CARVE-VLA reaches 92.5% (185/200). The gain is modest at the suite level because many tasks are near saturation, but it is meaningful because the backbone is unchanged and the improvement comes from inference-time process control.

Table 3. Main completed comparison on `libero_10`.

Method	Success Episodes
<code>pi05_libero</code>	90.0% 180/200
Refined full CARVE-VLA	92.5% 185/200

The per-task pattern in Fig. 5 and Table 4 is more informative than the aggregate alone. The dominant weak task improves from 55.0% to 75.0%. Most saturated tasks remain stable, while small task-dependent fluctuations persist. This supports a restrained interpretation: CARVE-VLA is not a uniformly beneficial wrapper, but a process-control layer that is most useful when the baseline has visible long-horizon weakness.

5.2 Weak-Task Diagnosis and Ablation

The weak-task subset is the most useful diagnostic regime because saturated tasks hide process-control differences. Table 5 summarizes the targeted follow-up on T6/T8/T9. After the original full-stack configuration exposed the weak

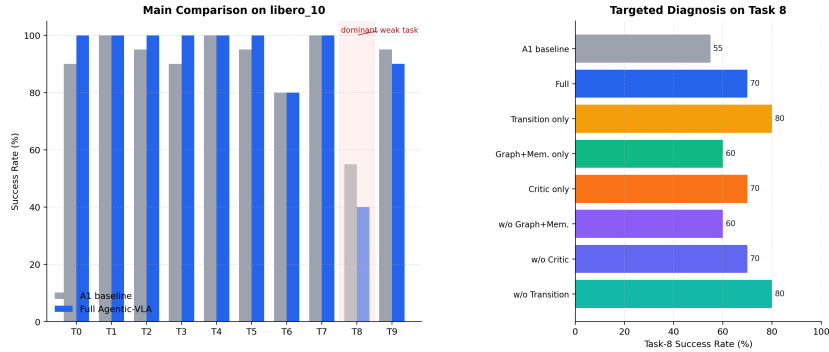


Fig. 5. Task-level comparison on `libero_10`. The main gain is concentrated in the weak-task region rather than distributed uniformly across saturated tasks.

Table 4. Per-task success on `libero_10`. Values are percentages.

Method	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
<code>pi05_libero</code>	90	100	95	90	100	95	80	100	55	95
Refined full CARVE-VLA	95	100	90	95	95	100	80	100	75	95

region, a task-targeted v2 follow-up reaches 90.0% on T6, 70.0% on T8, and 100.0% on T9. This run should be interpreted as targeted repair of exposed failure modes, not as a new suite-level claim.

Table 5. Targeted weak-task diagnosis. Values are percentages.

Method	T6	T8	T9
<code>pi05_libero</code>	80	55	95
Original full	80	40	90
Task-targeted v2	90	70	100
Transition only	100	80	90
Priors/memory only	80	60	100
Critic only	90	70	90

The ablations reveal two important points. First, scene priors and memory contribute positively on difficult cases: removing them drops T8 performance in the targeted setting. Second, module interaction is non-trivial. On T8, Transition-only reaches 80.0%, higher than the task-targeted full setting. This does not invalidate the framework; it indicates that the remaining bottleneck is coordination among useful modules rather than the absence of process control.

Mechanism traces support the same diagnosis. In the audited full-stack run, transition interventions average 0.81 per episode and intervened episodes succeed in 88.9% of cases. Critic-triggered retries are absent, so the critic is retained as an audit/escalation interface rather than credited with the observed benchmark gain.

5.3 Flow-Step and Backend Calibration

Table 6 separates policy sampling from action commitment. Reducing seven flow steps to two while keeping commitment at ten preserves 14/15 paired successes, lowers mean VLA-call latency from 366.9 to 149.0 ms (2.46 $\times$), and reduces episode wall time from 21.27 to 14.40 s (32.3%). The seemingly more adaptive 2/4-step controller reaches only 13/15. We therefore select the simple two-step profile instead of claiming that more online compute variation is automatically better.

Table 6. Paired flow-step calibration on 15 T6/T8/T9 episodes.

Profile	Success	Call (ms)	VLA/ep (s)	Wall/ep (s)
Reference: 7 steps, commit 10	14/15	366.9	11.15	21.27
Calibrated: 2 steps, commit 10	14/15	149.0	4.44	14.40
Dynamic: 2/4 steps, commit 10	13/15	158.9	4.98	15.91

The selected policy profile is then evaluated through the actual deployment boundary. Table 7 reports 50 measured calls after three warmups. Compiled BF16 reduces P95 from 159.59 to 67.40 ms and changes the 80-ms miss rate from 100% to 0%. SMVE reduces P95 by 16.6% further, to 56.19 ms. All three compiled candidates pass all 45 paired replay observations. W8A16 quantizes only VLM language layers 0–3, reducing peak VRAM from 6.98 to 6.56 GB relative to compiled BF16, but is not faster on this batch-one workload.

Table 7. RTX 4090 deployment profiles at two flow steps and horizon 10.

Profile	P50	P95	Miss	VRAM	Decision
Eager BF16	154.34	159.59	100%	7.12 GB	reference
Compiled BF16	65.73	67.40	0%	6.98 GB	default
Compiled BF16 + SMVE	54.35	56.19	0%	6.97 GB	padded-view default
Compiled W8A16	69.74	71.72	0%	6.56 GB	closed-loop rejected

The same contract is exercised on autoregressive OpenVLA-7B using ten paired T8/T9 frames. Eager BF16 reaches P50/P95 of 303.48/310.92 ms at 14.42 GB. INT8/NF4 lower VRAM to 7.76/4.41 GB, but are slower and match only

50%/10% of action vectors, so both are rejected. Decode accounts for 67.2% of CUDA time; exact token-mask alignment and prewarmed LLM compilation reduce P50/P95 to 224.25/231.84 ms with 10/10 exact actions. Preparation takes 200.19 s and all calls still exceed 100 ms, making this a scoped speedup, not a 10-Hz claim.

In the paired synchronous T8/T9 gate, SMVE reaches 8/10 versus 7/10 and reduces wall time from 205.79 to 199.12 s. This supports non-inferiority, not a task-competence improvement claim.

5.4 Physical Recovery and Closed-Loop Gating

Replay acceptance is necessary but not sufficient. From the same restored T6 stall state, compiled BF16 accurate replanning fails at the 280-step horizon, whereas a two-call Agentic retry succeeds in 235 steps and bounded physical recovery plus replan succeeds in 236. Repeating the physical branch reproduces success, step count, 112 VLA calls, 12 recovery actions, and final-state digest. The online sentinel further executes first-stall replanning, repeated-stall physical recovery, response verification, and post-recovery replanning in one controller lifecycle; it succeeds with a 0.0180-m end-effector response and zero policy-call deadline misses.

The W8A16 profile loses the compiled-BF16 T6 recovery success despite passing all replay checks and satisfying the latency deadline. It is therefore rejected by the closed-loop gate. This negative result is important for lightweight VLA deployment: small action error can accumulate into a different intervention outcome even when open-loop aggregate metrics appear acceptable. Compiled BF16 remains the default. On the corrected synchronous commit-10 held-out T8/T9 gate, the controller completes 2/2 tasks and verifies 1/1 physical recovery attempts.

5.5 Risk-Gated Asynchronous Execution

Model-call real time does not imply control-loop real time. In the synchronous T6 profile, image preparation and MuJoCo stepping raise a 69.09-ms policy P95 to 114.01-ms VLA-step control P95. A repeated single-state pilot verified that risk-gated prefetch can overlap this critical path; it is implementation evidence rather than the deployment decision.

The initial two-episode check preserved both successes, but Table 8 gives the decisive five-state-per-task expansion. Full-duty prefetch reduces deadline misses from 417/3931 to 48/4287 (89.4% relative), yet success drops from 7/10 to 5/10. Requiring a fresh synchronous chunk after every prefetched chunk still reaches only 5/10. The failures are not worker faults: both profiles have zero inference errors and verified physical recoveries, but temporally shifted observations alter the subsequent VLA trajectory. We therefore reject asynchronous prefetch from the promoted runtime. Compiled BF16 remains the fidelity-first default, and the residual end-to-end synchronization problem is left explicit rather than hidden behind model-call latency.

Table 8. Expanded T8/T9 asynchronous-execution gate (ten episodes).

Execution	Success	Calls	Wall (s)	80-ms miss	Miss rate
Sync, commit 8	7/10	417	205.79	417/3931	10.61%
Async, every chunk	5/10	346	198.21	48/4287	1.12%
Async, alternating	5/10	315	207.58	176/4278	4.11%

6 Discussion and Limitations

Supported claim. The completed LIBERO-10 comparison supports a conservative claim: a strong frozen VLA can be improved by an external inference-time process-control layer. The improvement is modest at the suite level because the baseline is already strong and many tasks are saturated, but the gain appears where the process-control hypothesis predicts it should: in exposed weak long-horizon tasks. The runtime studies support a complementary deployment claim: flow-step calibration, compilation, and contract-checked padding elimination preserve paired success while reducing model cost. The closed-loop rejection of both W8A16 and asynchronous scheduling shows why latency, memory, and replay fidelity cannot replace embodied evaluation.

What is not claimed. CARVE-VLA is not a new VLA backbone and does not claim universal benchmark dominance. The current runtime uses explicit risk- and deadline-conditioned scheduling; it does not claim a learned globally optimal compute-allocation policy or hard real-time guarantees. SMVE applies only when an adapter guarantees a static padding view. The OpenVLA result validates the shared adapter, profiling, and fidelity boundary, not transfer of this PI0.5-specific transform or a universal speedup. W8A16 is a standard deployment transform evaluated inside our gate, not a novel quantization algorithm, and its closed-loop rejection precludes a positive compressed-model claim. Likewise, physical recovery is supplied by the external execution loop rather than learned by the VLA. This distinction is central to the paper: the contribution is deciding how a frozen policy should be supervised during execution.

Ablation lesson. The weak-task ablations prevent overclaiming. Transition and priors/memory are useful, but their combination is not automatically optimal. The critic is aligned with failure recovery, yet under-activated in the tuned configuration. The next algorithmic problem is therefore module routing under roll-out uncertainty: when should the runtime activate Transition, memory, retry, or plain VLA execution?

Scope and next experiments. Benchmark claims are restricted to completed LIBERO-10 rollouts; the calibration, recovery, and prefetch studies are acceptance gates rather than benchmark-scale estimates. OpenVLA supplies cross-family runtime evidence, not a second closed-loop benchmark result; LIBERO-Plus remains necessary for a broader task claim. Key transfer risks are heuristic

overfitting, module interference, temporal observation shift, and robot-specific recovery skills.

7 Conclusion

We presented CARVE-VLA, a modular runtime that treats frozen-VLA deployment as a joint process-control and inference-efficiency problem. On LIBERO-10 it improves `pi05_libero` from 90.0% to 92.5%, while stateful recovery closes the repeated-stall controller lifecycle. The Optimize Runtime preserves paired success under two-step sampling, selects compiled BF16, and rejects quantization and asynchronous schedules when closed-loop outcomes regress. For padded-view inputs, SMVE passes both gates and reduces deployment P95 by 16.6% beyond compilation. Agentic reliability and deployment efficiency should therefore be validated as one embodied runtime rather than as disconnected modules.

Acknowledgments. Omitted for double-blind review.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Black, K., Brown, N., Driess, D., Esmail, A., et al.: π_0 : A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164 (2024), <https://arxiv.org/abs/2410.24164>
2. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., et al.: RT-1: Robotics transformer for real-world control at scale. arXiv preprint arXiv:2212.06817 (2022), <https://arxiv.org/abs/2212.06817>
3. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., et al.: RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818 (2023), <https://arxiv.org/abs/2307.15818>
4. Duan, J., Pumacay, W., Kumar, N., et al.: AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. arXiv preprint arXiv:2410.00371 (2024), <https://arxiv.org/abs/2410.00371>
5. Duan, Z., Zhang, Y., Geng, S., et al.: Fast ECoT: Efficient embodied chain-of-thought via thoughts reuse. arXiv preprint arXiv:2506.07639 (2025), <https://arxiv.org/abs/2506.07639>
6. Fan, Y., Ding, P., Bai, S., Tong, X., et al.: Long-VLA: Unleashing long-horizon capability of vision language action model for robot manipulation. arXiv preprint arXiv:2508.19958 (2025), <https://arxiv.org/abs/2508.19958>
7. Grislain, C., Rahimi, H., Sigaud, O.: I-FailSense: Towards general robotic failure detection with vision-language models. arXiv preprint arXiv:2509.16072 (2025), <https://arxiv.org/abs/2509.16072>
8. Guan, W., Hu, Q., Li, A., et al.: Efficient vision-language-action models for embodied manipulation: A systematic survey. arXiv preprint arXiv:2510.17111 (2025), <https://arxiv.org/abs/2510.17111>

9. Han, S., Qiu, B., Liao, Y., Huang, S., et al.: RoboCerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. arXiv preprint arXiv:2506.06677 (2025), <https://arxiv.org/abs/2506.06677>
10. Jin, R., Zhang, Z.: Agentic-VLA: Efficient online adaptation for vision-language-action models. arXiv preprint arXiv:2605.22896 (2026), <https://arxiv.org/abs/2605.22896>
11. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., et al.: OpenVLA: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246 (2024), <https://arxiv.org/abs/2406.09246>
12. Li, R., Zhou, Y., Zhu, Y., Chen, K., et al.: RoboClaw: An agentic framework for scalable long-horizon robotic tasks. arXiv preprint arXiv:2603.11558 (2026), <https://arxiv.org/abs/2603.11558>
13. Lin, Z., Duan, J., Fang, H., Fox, D., et al.: FailSafe: Reasoning and recovery from failures in vision-language-action models. arXiv preprint arXiv:2510.01642 (2025), <https://arxiv.org/abs/2510.01642>
14. Liu, B., Zhu, Y., Gao, C., Feng, Y., et al.: LIBERO: Benchmarking knowledge transfer for lifelong robot learning. arXiv preprint arXiv:2306.03310 (2023), <https://arxiv.org/abs/2306.03310>
15. Mees, O., Hermann, L., Rosete-Beas, E., Burgard, W.: CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. arXiv preprint arXiv:2112.03227 (2021), <https://arxiv.org/abs/2112.03227>
16. Niu, J., Gu, K., Zhao, Y., Liang, S., et al.: Realtime-VLA FLASH: Speculative inference framework for diffusion-based VLAs. arXiv preprint arXiv:2605.13778 (2026), <https://arxiv.org/abs/2605.13778>
17. Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., et al.: Octo: An open-source generalist robot policy. arXiv preprint arXiv:2405.12213 (2024), <https://arxiv.org/abs/2405.12213>
18. Open X-Embodiment Collaboration, O'Neill, A., Rehman, A., Gupta, A., et al.: Open x-embodiment: Robotic learning datasets and RT-X models. arXiv preprint arXiv:2310.08864 (2023)
19. Pang, Y., Zhou, B., Li, C., Wang, X., et al.: Sci-VLA: Agentic VLA inference plugin for long-horizon tasks in scientific experiments. arXiv preprint arXiv:2602.09430 (2026)
20. Physical Intelligence, Black, K., Brown, N., Darpinian, J., et al.: $\pi_{0.5}$: A vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054 (2025)
21. Shi, H., Xie, B., Liu, Y., Sun, L., et al.: MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. arXiv preprint arXiv:2508.19236 (2025)
22. Torne, M., Pertsch, K., Walke, H., Vedder, K., et al.: MEM: Multi-scale embodied memory for vision language action models. arXiv preprint arXiv:2603.03596 (2026)
23. Xu, S., Wang, Y., Xia, C., Wu, D., et al.: VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching. arXiv preprint arXiv:2502.02175 (2025)
24. Yang, C., Hu, Y., Ma, Y., Yang, Y., et al.: Realtime-VLA v2: Learning to run VLAs fast, smooth, and accurate. arXiv preprint arXiv:2603.26360 (2026)
25. Yang, Z., Chen, Y., Zhou, X., Yan, J., et al.: Agentic robot: A brain-inspired framework for vision-language-action models in embodied agents. arXiv preprint arXiv:2505.23450 (2025)
26. Zawalski, M., Chen, W., Pertsch, K., Mees, O., et al.: Robotic control via embodied chain-of-thought reasoning. arXiv preprint arXiv:2407.08693 (2024)